Microsoft Purview Executive Dashboard

Implementation Solution Guide

Version 1.0.0 · 2026-07-07

Contents

Executive Summary — Microsoft Purview Executive Dashboard 5

The problem . . . 5

The solution . . . 5

What you get . . . 5

Approach . . . 5

Boundaries . . . 6

The bottom line . . . 6

1. Solution Overview — Microsoft Purview Executive Dashboard 6

What this solution is . . . 6

Why this matters . . . 6

How it works . . . 7

The signal stack this dashboard summarizes . . . 7

What you can control . . . 7

Limitations (be honest with buyers) . . . 8

Naming convention (used in every doc and script) . . . 8

The standard deployment model . . . 9

2. Requirements Guide — Microsoft Purview Executive Dashboard 9

2.1 Compliance/data-governance licensing for the signals you want to show . . . 9

2.2 Least-privilege reader roles across the signal sources . . . 9

2.3 Security & Compliance PowerShell + Graph toolchain . . . 10

2.4 An agreed metric set and thresholds (from compliance stakeholders) . . . 10

2.5 In-scope workloads, labels, DLP policies, and assessments (for coverage numbers) . . . 10

2.6 Portal-read values and who provides them (Compliance Manager score, Insider-Risk aggregate, DSPM) . . . 10

2.7 A render/distribution target (HTML/CSV, optional Power BI) . . . 11

2.8 A pilot draft, refresh cadence, and change window . . . 11

Global Requirements — Microsoft Purview Solutions 11

1. Microsoft 365 tenant with the required Purview/compliance licensing . . . 12

2. Purview roles (least-privilege) and portal access . . . 12

3. Security & Compliance PowerShell + Graph toolchain . . . 12

4. Data sources & scope in scope . . . 12

5. Compliance, records, privacy & legal stakeholders . . . 13

6. A pilot scope and change control . . . 13

3. Implementation Guide — Microsoft Purview Executive Dashboard 13

Step 1 — Assess the available signals and licensing (Assess) . . . 14

Step 2 — Design the metric set and thresholds (Design) . . . 14
Step 3 — Configure the definition and collection (Configure) . . . 14
Step 4 — Collect and build the pilot one-pager (Configure → Pilot) . . . 15
Step 5 — Reconcile every number against its Purview source (Pilot) . . . 15
Step 6 — Validate the page (Validate) . . . 16
Step 7 — Publish, schedule, and accrue trend (Expand) . . . 16
Step 8 — Operate and keep it trustworthy (Expand) . . . 16
5. Testing & Validation — Microsoft Purview Executive Dashboard 16
Gate 1 — Sources are reachable and licensing is known . . . 17
Gate 2 — The metric definition is complete and approved . . . 17
Gate 3 — The pilot one-pager renders with all three sections . . . 17
Gate 4 — Every number reconciles to its Purview source . . . 17
Gate 5 — Coverage denominators are correct . . . 17
Gate 6 — Portal-only values are current (not stale) . . . 18
Gate 7 — Insider Risk is aggregate and de-identified . . . 18
Gate 8 — Thresholds colour correctly . . . 18
Gate 9 — Trend accrues across refreshes . . . 18
Gate 10 — Scheduled refresh and distribution work end to end . . . 18
Evidence to retain . . . 19
6. Troubleshooting — Microsoft Purview Executive Dashboard 19
A KPI shows blank / "no data" . . . 19
A number doesn't match the Purview portal . . . 19
The compliance score / Insider-Risk aggregate / DSPM count is stale . . . 20
`Export-ContentExplorerData` / `Export-ActivityExplorerData` errors or returns nothing . . . 20
Trend section is empty or shows a sudden step-change . . . 20
The scheduled collection didn't run . . . 20
Insider Risk detail appears on the page (privacy incident) . . . 20
Connect-IPPSSession / Connect-MgGraph fails . . . 21
Timestamps / percentages look inconsistent across sources . . . 21
When to escalate beyond this solution . . . 21
7. Best Practices — An Executive Dashboard the C-Suite Trusts 21
Report the truth, don't decorate it . . . 21
Trend is the executive signal — protect the history . . . 22
Least-privilege, read-only, and Insider-Risk role separation . . . 22
Thresholds are a governance decision, not a default . . . 22
Respect the licensing and preview reality . . . 22

Validate before every distribution . . . 22
Keep it a one-pager . . . 23
8. Deliverables Summary — Microsoft Purview Executive Dashboard 23
Documentation . . . 23
PowerShell tools (5) . . . 24
Named artifacts created . . . 25
Definition of done . . . 25
Appendix A — PowerShell Scripts 26

Executive Summary — Microsoft Purview Executive Dashboard

The problem

Boards, audit committees, regulators, and cyber-insurers now ask the same standing question: *how compliant are we, how exposed is our data, and are we getting better or worse?* Microsoft Purview holds the answer — but scattered across Compliance Manager, DLP, sensitivity labels, Content and Activity Explorer, Insider Risk, DSPM, and Data Lifecycle/Records, each its own blade on its own refresh. There is no single C-suite one-pager, and nothing shows **trend** unless someone captures snapshots over time. So every time the question is asked, the answer is a slow, manual scramble — different each time, and impossible to trend.

The solution

A complete, self-guided **executive dashboard** for Microsoft Purview: **one page** covering **compliance posture, data risk, and trend**. It defines the metric set and thresholds with your stakeholders, collects the signals with safe read-only PowerShell (Security & Compliance PowerShell / Microsoft Graph), merges the few portal-only values, renders the one-pager (HTML/CSV, Power BI-ready), pilots it as a **draft** with every number checked against its Purview source, validates that it reconciles, and schedules it to refresh — capturing a **monthly snapshot** so the page shows direction, not just a point. Five safe-by-default PowerShell tools (read-only, or `-WhatIf` for the only two writes — the definition file and the schedule) do the assessment, configuration, collection/render, validation, and scope/health checks.

What you get

- **Compliance posture, at a glance** — Compliance Manager score and delta, outstanding improvement actions, active regulatory assessments.
- **Data risk, quantified** — sensitivity-label coverage, high-severity DLP volume, sensitive items detected, retention coverage, and (where licensed) Insider Risk aggregate alerts and DSPM open recommendations.
- **Trend, not just a point** — a monthly snapshot so posture, DLP volume, label coverage, and incident counts are shown moving — the thing executives actually act on.
- **Every number attributed** — each KPI shows its data source and as-of timestamp, reconciled to its Purview blade, with unlicensed sources honestly flagged rather than fabricated.
- **Trustworthy by design** — pilot-first, validated before distribution, Insider Risk aggregate/de-identified with strict role separation.

Approach

Delivery follows a disciplined, reversible model — **Assess** → **Design** → **Configure** → **Pilot** → **Validate** → **Expand** — building and reconciling the one-pager as a **draft** before any executive sees it, with pass/fail gates and documented rollback. The dashboard is a **reporting layer**: it changes no Purview policy; its only artifacts are a definition file, the snapshots, and a scheduled collection task.

Boundaries

The signals are **licensing-/plan-dependent** (DSPM, Insider Risk, Content Explorer, and advanced DLP require E5 / E5 Compliance; some preview) and the portal is **actively evolving** — capabilities and paths are cited or flagged **"verify your licensing/tenant,"** never assumed. Some values are **portal-only** in many tenants and are read by a named owner, not scraped. Whether the posture the dashboard reports is **adequate**, and any **regulatory/records/legal determination**, are **deferred to your compliance, records, privacy, and legal functions** — the solution provides the accurate measurement, not the judgement.

The bottom line

A one-page view for the C-suite — compliance posture, data risk, and trend — **configured, reconciled, validated, and defensible** in Microsoft Purview, so the next time the board asks, the answer is already on the page.

1. Solution Overview — Microsoft Purview Executive Dashboard

What this solution is

Executives don't want to read Activity Explorer, click through Compliance Manager, and cross-reference DLP alerts — they want **one page** that answers three questions: *how compliant are we, how exposed is our data, and are we getting better or worse?* Microsoft Purview holds all of that signal, but it is scattered across Compliance Manager, Data Loss Prevention, sensitivity labels, Content/Activity Explorer, Insider Risk, Data Security Posture Management (DSPM), and Data Lifecycle/Records — each its own portal blade, each on its own refresh. This solution assembles those signals into a **single, repeatable, one-page executive dashboard** covering **compliance posture, data risk, and trend**. It defines the metric set, collects the signals with safe read-only PowerShell (Security & Compliance PowerShell / Microsoft Graph), renders the one-pager (HTML/CSV, ready for Power BI), pilots it in a **draft** before it goes to the C-suite, validates every number against its source, and schedules it to refresh — so the capability is in place and defensible. It uses only the **Microsoft Purview portal**, **Security & Compliance PowerShell**, and **Microsoft Graph** — no third-party tooling.

Why this matters

- **Boards and executives are accountable for data risk.** "Show me our compliance posture" is now a standing question from audit committees, regulators, and cyber-insurers. Without a curated view, the answer is a slow, manual scramble across a dozen Purview blades — different every time it's asked.
- **The signal already exists but isn't executive-ready.** Compliance Manager produces a score; DLP produces alerts; labels and Content Explorer produce coverage; Insider Risk and DSPM produce risk indicators. None of them, on their own, is a C-suite one-pager, and none of them shows **trend** without you capturing snapshots over time.
- **Trend is the executive's real question.** A single-point score is noise; *direction* is signal. This solution captures a **monthly snapshot** so posture, DLP volume, label coverage, and incident counts

can be shown moving — the thing an executive actually acts on.

• **The sources are licensing-dependent and evolving.** DSPM, Insider Risk, Content Explorer, and advanced DLP are E5 / E5 Compliance (some preview); Compliance Manager's baseline is broader. The dashboard must show what your tenant **actually** licenses and flag the rest — not fabricate a number.

How it works

1. **Assess the sources.** Discovery connects to Security & Compliance PowerShell and Graph and reports which signals are reachable and licensed — Compliance Manager visibility, DLP policies, published labels, Content/Activity Explorer access, retention coverage, and whether Insider Risk / DSPM are licensed (E5).

2. **Define the metric set.** A single **metric definition** names every KPI on the page, its source, its refresh, and its **thresholds** (green/amber/red) — agreed with your compliance stakeholders, not invented by the tool.

3. **Collect safely and read-only.** The collector pulls current values from each source cmdlet/API into a normalized **snapshot** (a file — no tenant change), and merges the few portal-only values (Compliance Manager score, Insider-Risk aggregate, DSPM recommendations) that Purview does not expose to automation in your tenant.

4. **Pilot in a draft.** The one-pager is built and reviewed as `Exec Dashboard – Pilot – Stage1` **first** — every number checked against its Purview source — before any executive sees it.

5. **Validate every number.** A validation pass confirms each section populates, cross-checks counts against source, and flags **stale** or **missing** data so the page is never silently wrong.

6. **Render, snapshot, and schedule.** The report renders the one-page dashboard (HTML/CSV, Power BI-ready), appends the month's values to the **trend baseline**, and the collection is scheduled so the page refreshes and trend accrues automatically.

The signal stack this dashboard summarizes

• **Compliance posture** — **Compliance Manager** score & improvement actions; active **regulatory assessments**.

• **Data risk** — **sensitivity label** coverage; **DLP** high-severity match volume; **sensitive items** detected (Content Explorer); **retention/records** coverage; and, where licensed, **Insider Risk** aggregate alerts and **DSPM** open recommendations.

• **Trend** — a **monthly snapshot** of the above so the page shows direction, not just a point.

What you can control

• The **metric set** and its **thresholds** (what counts as green/amber/red for each KPI).

• Which **workloads, labels, DLP policies, and assessments** are in scope for the coverage numbers.

• The **refresh cadence** and the **distribution** (who receives the one-pager, in what format).

• The **pilot draft** validated before the C-suite version is published.

• How **privacy-sensitive** signals (Insider Risk) are represented — **aggregate only, no names** (see limitations).

Limitations (be honest with buyers)

• **This is a reporting/summarization layer, not a new control.** It surfaces posture and risk from Purview's existing capabilities; it does not itself label, block, or investigate. If DLP, labels, or Insider Risk aren't configured, the dashboard will honestly show low coverage — fix the underlying capability, not the report.

• **Several sources are licensing-gated (E5 / E5 Compliance; some preview).** DSPM, Insider Risk, Content Explorer, and advanced DLP require the right plan; Compliance Manager's premium assessments do too. The dashboard shows what your tenant **actually** has and flags the rest as *"verify your licensing/tenant"* — it never invents a value.

• **Some values are portal-only in many tenants.** Microsoft does not expose a stable public automation surface for the Compliance Manager **score**, the Insider Risk **aggregate**, or DSPM **recommendations** in every tenant; where that's the case those are **read from the portal and entered into the metric definition** (with the read date recorded), not scraped. **Verify the current supported method for your tenant.**

• **Trend requires history you must accrue.** The trend section is only as deep as the snapshots you've captured — it starts building the day you run the first collection, not retroactively.

• **Insider Risk is HR/privacy-sensitive.** Only **aggregate, de-identified** counts appear on an executive page — never user names or case detail. Access follows strict **role separation** (Insider Risk roles are distinct from dashboard authors), and outcomes are **deferred to HR/legal**.

• **Regulatory/records/legal determinations are not ours to make.** Whether a compliance score, retention coverage, or an assessment set is *adequate* is a **compliance/records/privacy/legal** determination — the dashboard reports the technical measurement, not the judgement.

Naming convention (used in every doc and script)

Every artifact this solution creates is named `<Purpose> – <Scope> – <Stage>` and reused verbatim across the docs, scripts, and portal. The standard set:

Artifact	Name
The one-page executive dashboard	`Exec Dashboard – Posture-Risk-Trend – OnePage`
The metric definition (KPIs, sources, thresholds, scope)	`Exec Metrics – Compliance Signals – Definition`
The scheduled collection job	`Exec Collector – Purview Signals – Scheduled`
The monthly trend store	`Exec Trend – Monthly Snapshot – Baseline`
The distribution of the one-pager	`Exec Dashboard – C-Suite – Distribution`
The pilot draft validated before publish	`Exec Dashboard – Pilot – Stage1`

The standard deployment model

> *Assess which Purview signals you actually have and license → design the metric set and thresholds → configure the definition, collection, and schedule → **pilot** the one-pager as a draft and check every number against its source → validate it populates and reconciles → expand to scheduled refresh, trend accrual, and C-suite distribution. Pattern: **Assess** → **Design** → **Configure** → **Pilot** → **Validate** → **Expand.***

2. Requirements Guide — Microsoft Purview Executive Dashboard

Verify **every** requirement before building the dashboard. Each entry: **Description · Why · Verify · Implement-if-missing.** Category-wide prerequisites (tenant/licensing, Purview roles, the S&C PowerShell + Graph toolchain, in-scope workloads, compliance/records/legal stakeholders, pilot & change control) are in `global-requirements-purview.md` — this guide adds the dashboard-specific detail.

> ***Licensing/tenant boundary.** The signals this dashboard summarizes are **licensing-gated**: **DSPM**, **Insider Risk**, **Content Explorer**, and advanced **DLP** require **Microsoft 365 E5** or the **E5 Compliance** add-on (some are in **preview**); **Compliance Manager** and baseline labels/ DLP/retention are broader but premium assessments are still gated. Some values (Compliance Manager **score**, Insider Risk **aggregate**, DSPM **recommendations**) have **no stable public automation surface in every tenant** and are read from the portal. Where this guide flags *"verify your licensing/tenant,"* do so before relying on the metric — the dashboard shows what you have, it never invents a number.*

2.1 Compliance/data-governance licensing for the signals you want to show

Description: The dashboard's coverage depends on what's licensed: **Compliance Manager** (baseline included broadly; premium assessments = E5/E5 Compliance), **sensitivity labels** and **retention** (E3 baseline; auto-labeling/advanced = E5), **DLP** (E3 baseline; advanced classifiers/analytics = E5), **Content Explorer** (E5), **Insider Risk** and **DSPM** (E5/E5 Compliance; some preview). **Why:** A KPI can only be populated where its source capability is licensed; unlicensed sources are shown as *"not licensed / verify tenant,"* never fabricated (GR §1). **Verify:** M365 admin center → `Billing → Licenses`; and run `OPSEXEC-Discovery.ps1`, which reads `subscribedSkus`/service plans via Graph and reports which signal sources are licensed/reachable. **Implement:** Assign/purchase the plan for any source you require on the page (e.g. E5 Compliance for DSPM/Insider Risk/Content Explorer) before expecting that KPI to populate.

2.2 Least-privilege reader roles across the signal sources

Description: Read access to each source: **Compliance Administrator** or **Compliance Data Administrator** (or the narrower **Global Reader**) for Compliance Manager, labels, DLP, and retention inventory; **Content Explorer** requires **Content Explorer List/Content Viewer** roles; **Insider Risk** requires an **Insider Risk Management** reader role held by a *separate* person (role separation). The dashboard author needs **read** across the sources — not write, not investigation access. **Why:** Purview uses fine-grained role groups; a reader-only dashboard needs the matching read roles, and Insider Risk is deliberately walled off (GR §2, §5). **Verify:** Purview portal → `Settings → Roles & scopes / Permissions` — confirm the read roles; `OPSEXEC-Scope.ps1` reports a permissions error against any

source the account can't read. **Implement:** Assign the least-privilege **read** roles under change control; keep **Insider Risk** roles separate from the dashboard-author role (the author sees only aggregate counts, never case detail).

2.3 Security & Compliance PowerShell + Graph toolchain

Description: The `ExchangeOnlineManagement` module for `Connect-IPPSSession` (Security & Compliance PowerShell — `Get-Label`, `Get-DlpCompliancePolicy`, `Get-RetentionCompliancePolicy`, `Export-ActivityExplorerData`, `Export-ContentExplorerData`), and `Microsoft.Graph` for licensing reads and any Graph reporting/security surface used. **Why:** The scripts collect signals through these endpoints (GR §3). **Note:** `Export-ContentExplorerData` and some reports are **preview/tenant-dependent** — confirm availability. **Verify:** `Get-Module -ListAvailable ExchangeOnlineManagement`; `Connect-IPPSSession` and `Connect-MgGraph` succeed with a modern-auth/MFA admin. `OPSEXEC-Discovery.ps1` connects and reports which endpoints/cmdlets are reachable. **Implement:** `Install-Module ExchangeOnlineManagement -Scope CurrentUser`; `Install-Module Microsoft.Graph -Scope CurrentUser`.

2.4 An agreed metric set and thresholds (from compliance stakeholders)

Description: A documented statement — **from your compliance, records, privacy, and security functions** — of *which* KPIs belong on the executive page and what **green/amber/red thresholds** mean for each (e.g. what compliance score, what DLP volume, what label coverage is "acceptable"). **Why:** Whether a number is *good* is a governance judgement, not a technical default (GR §5); the dashboard reports the measurement and colours it against **agreed** thresholds — it doesn't decide what's adequate. **Verify:** You can name each KPI, its source, its owner, and its threshold band; captured in `Exec Metrics – Compliance Signals – Definition`. **Implement:** Run a short workshop with the stakeholders to agree the metric set and thresholds before building the page.

2.5 In-scope workloads, labels, DLP policies, and assessments (for coverage numbers)

Description: The **denominator** for every coverage KPI: which workloads (Exchange, SharePoint, OneDrive, Teams, endpoints) count toward retention/label coverage, which **published labels** and **DLP policies** are the in-scope set, and which **Compliance Manager assessments** are the ones executives care about. **Why:** "Label coverage = 60%" is meaningless without a defined scope; the coverage percentages key off the in-scope set (GR §4). **Verify:** `OPSEXEC-Scope.ps1` lists published labels, DLP policies, retention policies, and workloads so you can confirm the in-scope set; the numbers are recorded in the metric definition. **Implement:** Define the in-scope workloads/labels/policies/assessments with stakeholders before computing coverage.

2.6 Portal-read values and who provides them (Compliance Manager score, Insider-Risk aggregate, DSPM)

Description: Values that many tenants can't pull via a stable public API — the **Compliance Manager score**, the **Insider Risk aggregate** alert count by severity, and the **DSPM open recommendations** count — are **read from the Purview portal** by the appropriately-roled owner and entered into the metric

definition, **with the read date recorded**. **Why:** These are portal-primary in many tenants; the dashboard must not scrape or invent them, and Insider Risk must stay **aggregate/de-identified** with **role separation** (GR §2, §5). **Verify:** The metric definition has a named owner and a read date for each portal-only value; `OPSEXEC-Validate.ps1` flags any portal-only value that is **stale** (older than your refresh window). **Implement:** Assign an owner for each portal-only value and agree how often they refresh it; **verify the current supported automation method for your tenant** in case an API is now available.

2.7 A render/distribution target (HTML/CSV, optional Power BI)

Description: Where the one-pager is produced and delivered — the scripts emit **HTML + CSV** by default (self-contained, no external calls); optionally the CSV/snapshot feeds a **Power BI** report for a richer visual. Distribution is `Exec Dashboard – C-Suite – Distribution` (e.g. a secured SharePoint location or an emailed PDF). **Why:** The deliverable is a *page*, so a defined output/format and a secured distribution channel are needed; executives receive a summary, not portal access. **Verify:** The output folder and distribution channel are agreed; `OPSEXEC-Report.ps1` writes the HTML/CSV to the target and (optionally) the Power BI-ready dataset. **Implement:** Choose the output location and distribution method (and Power BI, if used) before scheduling the refresh.

2.8 A pilot draft, refresh cadence, and change window

Description: A **pilot** — the one-pager built as `Exec Dashboard – Pilot – Stage1` and every number checked against its source **before** any executive sees it — plus an agreed **refresh cadence** (typically monthly for trend, with an on-demand rebuild) and a change window for scheduling the collection (GR §6). **Why:** An executive dashboard that's wrong once loses trust permanently; validate small first, then schedule (GR §6 discipline). **Verify:** A named pilot draft and refresh cadence exist, and the scheduled-collection change is approved. **Implement:** Define the pilot, cadence, and change window before publishing or scheduling anything.

Global Requirements — Microsoft Purview Solutions

These requirements apply to **every** solution in the **Purview** category (implementing and managing Microsoft Purview, sensitivity & retention labels, DLP, data lifecycle & records, insider risk, communication compliance, audit, data security posture management, and data governance). Verify each before starting a solution's implementation steps.

> ***Vendor boundary (applies to every Purview solution).*** *The **Microsoft Purview portal** (purview.microsoft.com), the **Microsoft 365 admin center**, **Security & Compliance PowerShell** (`Connect-IPPSSession`), and **Microsoft Graph** are authored at exact, current click-by-click / cmdlet detail from Microsoft's public documentation. Microsoft Purview features are heavily **licensing-/plan-dependent** (many require **Microsoft 365 E5** or the **E5 Compliance** add-on; some are in preview) and the portal is **actively evolving** — capability availability and portal paths are **cited or flagged "verify your licensing/tenant"**, never asserted or invented. **Regulatory, records-retention, and legal determinations** (retention schedules, records file plans, legal holds, insider-risk case actions, regulatory applicability) are **deferred to the organization's compliance, records, privacy, and legal functions** — the solutions provide the technical configuration method and evidence, not the determination. Automation is **PowerShell** (Security & Compliance PowerShell / Graph) — never with hardcoded credentials or secrets.*

1. Microsoft 365 tenant with the required Purview/compliance licensing

- **Description:** A Microsoft 365 tenant with the **compliance licensing** the capability requires

(E3 for baseline; **E5 / E5 Compliance** for advanced DLP, auto-labeling, insider risk, communication compliance, Audit Premium, DSPM, etc.).

- **Why required:** Most Purview capabilities are plan-gated; the feature won't be available without the

right plan.

- **How to verify:** In the M365 admin center confirm the subscription/add-ons; check the feature is

visible in the Purview portal.

- **How to implement if missing:** Assign/purchase the required licensing before starting.

2. Purview roles (least-privilege) and portal access

- **Description:** Access with the least-privilege **Purview / compliance role** the solution names

(e.g. Compliance Administrator, Compliance Data Administrator, Information Protection, Records Management, Insider Risk, eDiscovery, Audit roles), with Global Administrator only where required.

- **Why required:** Purview uses fine-grained role groups; configuration requires the matching role.
- **How to verify:** Purview portal → **Roles & scopes / Permissions** — confirm the role membership.
- **How to implement if missing:** Assign the least-privilege role group under change control.

3. Security & Compliance PowerShell + Graph toolchain

- **Description:** A workstation with **Security & Compliance PowerShell** (`Connect-IPPSSession` via the

ExchangeOnlineManagement module) and **Microsoft Graph PowerShell**, modern-auth / MFA-capable admin.

- **Why required:** Configuration, automation, and the provided scripts run through these.
- **How to verify:** `Connect-IPPSSession` and `Connect-MgGraph` succeed with the admin account.
- **How to implement if missing:** Install the modules and confirm modern-auth sign-in before running steps.

4. Data sources & scope in scope

- **Description:** The **workloads and data sources** in scope (Exchange, SharePoint, OneDrive, Teams,

endpoints/devices, and — for data governance — the broader data estate) and a **data-classification / sensitivity** intent to drive labels, DLP, and retention.

- **Why required:** Purview policies key off the in-scope workloads and the classification scheme.
- **How to verify:** Confirm the in-scope workloads and the classification/sensitivity approach with

stakeholders.

- **How to implement if missing:** Define the scope and classification intent before authoring policies.

5. Compliance, records, privacy & legal stakeholders

- **Description:** Engagement with the **compliance, records-management, privacy, security, and legal**

stakeholders whose policies the configuration must satisfy (retention schedules, records file plans, regulatory obligations, insider-risk and communication-compliance policy, legal hold).

- **Why required:** Purview configuration must align to enterprise policy and legal/records requirements,

not technical defaults; several capabilities have HR/legal sensitivity.

- **How to verify:** Confirm the applicable policies, the records schedule, and named approvers.
- **How to implement if missing:** Identify the owners and capture the requirements before configuring.

6. A pilot scope and change control

- **Description:** A **pilot scope** (users/sites/policies in simulation or a limited ring) to validate

on before tenant-wide enforcement, and a **change-control** process — most Purview policies support a **simulation / test / disabled** mode that MUST be used before enforcement.

- **Why required:** Every solution follows an **Assess → Design → Configure → Pilot (simulate) → Validate

→ Expand (enforce)** model; DLP, labels, retention, and insider-risk changes are high-impact.

- **How to verify:** Confirm a named pilot scope, that simulation/test mode is available, and a change

window/approval path.

- **How to implement if missing:** Define the pilot and agree change control before enforcing anything.

*The deployment model across every Purview solution is the same: **Assess** → **Design** → **Configure** → **Pilot (simulate/test)** → **Validate** → **Expand (enforce)**, with explicit pass/fail gates and a documented rollback. Automation is delivered as **PowerShell** (Security & Compliance PowerShell / Graph) — never with hardcoded credentials or secrets. Regulatory/records/legal determinations are deferred to the relevant governance functions.*

3. Implementation Guide — Microsoft Purview Executive Dashboard

Follow these steps **in order**. Work uses the **Microsoft Purview portal** (`https://purview.microsoft.com`), **Security & Compliance PowerShell** (`Connect-IPPSSession` — `Get-Label`, `Get-DlpCompliancePolicy`, `Get-RetentionCompliancePolicy`, `Export-ActivityExplorerData`, `Export-ContentExplorerData`), and **Microsoft Graph** (licensing, any reporting/security surface). Every write is gated: scripts run **read-only or `-WhatIf` by default** and only change state (the metric definition file and the scheduled task) with `-Commit`. Artifact names follow the `<Purpose> – <Scope> – <Stage>` convention from the Overview and are reused verbatim.

__Model:__ Assess the available signals, design the metric set/thresholds, configure the definition and schedule, __pilot__ the one-pager as a draft and check every number against its source, validate it reconciles, then expand to scheduled refresh, trend accrual, and distribution. Pattern: __Assess__ → __Design__ → __Configure__ → __Pilot__ → __Validate__ → __Expand.__ The dashboard is a __reporting layer__ — it changes no Purview policy; the only artifacts it creates are the definition file, the snapshots, and the scheduled collection task.

Step 1 — Assess the available signals and licensing (Assess)

1. Run `OPSEXEC-Discovery.ps1` (read-only). It connects to S&C PowerShell and Graph and reports, per source:

• **Compliance posture** — Compliance Manager visible? active assessments count (portal-confirmed if no API);

• **Data risk** — published labels (`Get-Label`), DLP policies (`Get-DlpCompliancePolicy`), retention policies (`Get-RetentionCompliancePolicy`), Content/Activity Explorer reachable;

• **Licensing** — whether **DSPM**, **Insider Risk**, **Content Explorer**, advanced **DLP** service plans are present (Graph `subscribedSkus`).

1. Run `OPSEXEC-Scope.ps1` (read-only) to confirm connection/permission health across every source and list the candidate labels/policies/workloads that will form the in-scope set.

2. Record the gaps: sources not licensed, sources with no read role, values that are portal-only in your tenant. **Pass gate:** you have a written source inventory with, for each KPI, *reachable-by-API* vs *portal-read* vs *not licensed*.

Step 2 — Design the metric set and thresholds (Design)

1. With your compliance/records/security stakeholders (Requirement 2.4), agree the KPIs for the three sections and their green/amber/red thresholds. The standard set:

• **Posture:** `Compliance Manager Score` (% + points, Δ vs last month); `Improvement Actions Outstanding` (by High/Medium/Low); `Regulatory Assessments Active` (count).

• **Data risk:** `Sensitivity Label Coverage` (% of in-scope items labeled; labels published); `DLP High-Severity Matches (30d)` (count + Δ); `Sensitive Items Detected` (Content Explorer — items with SITs; oversharing indicator); `Retention Coverage` (% in-scope workloads with a retention policy); and, where licensed, `Insider Risk Active Alerts` (aggregate by severity — **no names**) and `DSPM Recommendations Open` (count).

• **Trend:** monthly snapshots of `Compliance Score`, `DLP Matches`, `Labeled-Content`, and `Insider-Risk Alerts`.

1. Fix the **in-scope set** (Requirement 2.5): which workloads/labels/DLP policies/assessments count toward coverage.

2. Name the owners and read cadence for the **portal-only** values (Requirement 2.6). **Pass gate:** the metric set, thresholds, in-scope set, and portal-value owners are documented and approved as `Exec Metrics – Compliance Signals – Definition`.

Step 3 — Configure the definition and collection (Configure)

1. Materialize the agreed design into the definition file, **pilot-first** (`-WhatIf` prints the exact file it would write):

`OPSEXEC-Configure.ps1 -DefinitionPath .\ExecMetrics.json -Scope <workloads/labels/policies> -Thresholds <bands> -WhatIf` → review → re-run with `-Commit`.

1. Enter (or reference) the **portal-only** values and their read dates in the definition (Compliance Manager score, Insider-Risk aggregate, DSPM recommendations) — keeping Insider Risk **aggregate/de-identified**.

2. Register the scheduled collection **in draft** — `OPSEXEC-Configure.ps1 -RegisterSchedule -Cadence Monthly -WhatIf` prints the scheduled-task command; **do not** `-Commit` the schedule until the pilot passes (Step 6). **Pass gate:** the definition file exists and validates; the schedule command is reviewed but not yet live. **Rollback:** delete the definition file / `-RegisterSchedule -Remove -Commit`.

Step 4 — Collect and build the pilot one-pager (Configure → Pilot)

1. Run the collector/renderer read-only against the pilot draft:

`OPSEXEC-Report.ps1 -DefinitionPath .\ExecMetrics.json -OutputPath .\ExecDashboard -Stage Pilot` It collects each API-reachable signal, merges the portal-only values from the definition, renders `Exec Dashboard – Pilot – Stage1` (HTML + CSV), and writes a snapshot.

1. The page shows every KPI with its value, its threshold colour, and a **data-source + as-of timestamp** per number — so nothing is unattributed.

2. **Pass gate:** the pilot one-pager renders with all three sections populated (or an explicit *"not licensed / portal-read pending"* where a source is unavailable). **Rollback:** none needed — output files only; delete and re-run.

Step 5 — Reconcile every number against its Purview source (Pilot)

1. For each KPI, open the source blade in the Purview portal and confirm the dashboard value matches:

• Compliance score → **Compliance Manager** dashboard; assessments → its assessments list.

• Label coverage / sensitive items → **Data classification** → **Content Explorer** (and `Export-ContentExplorerData`).

• DLP matches → **Data loss prevention** → **Alerts / Activity Explorer** (`Export-ActivityExplorerData`).

• Retention coverage → **Data lifecycle management / Records management** policies.

• Insider Risk aggregate → **Insider Risk Management** → **Alerts** dashboard (aggregate only).

• DSPM → **DSPM** recommendations (E5/preview).

1. Fix any mismatch at its cause: wrong scope/denominator, stale portal value, a filter difference, or an unlicensed source being counted. **Pass gate:** every number on the pilot page reconciles to its source, or is honestly labeled *"verify licensing/tenant."*

Step 6 — Validate the page (Validate)

1. Run `OPSEXEC-Validate.ps1 -DefinitionPath .\ExecMetrics.json -SnapshotPath .\ExecDashboard\snapshot.json` — it confirms every defined KPI is present, cross-checks the counts it can recompute, and flags **stale** portal-only values (older than the refresh window) and **missing** sources.

2. Walk the **Testing Guide** gates end to end. **Pass gate:** validation is green (no missing/stale KPIs) for the pilot.

Step 7 — Publish, schedule, and accrue trend (Expand)

1. Once the pilot reconciles and validates, produce the C-suite version:

`OPSEXEC-Report.ps1 -DefinitionPath .\ExecMetrics.json -OutputPath .\ExecDashboard -Stage Publish` → renders `Exec Dashboard – Posture-Risk-Trend – OnePage`.

1. Commit the schedule: `OPSEXEC-Configure.ps1 -RegisterSchedule -Cadence Monthly -Commit` (registers `Exec Collector – Purview Signals – Scheduled`). Each run appends to `Exec Trend – Monthly Snapshot – Baseline`, so the trend section deepens month over month.

2. Deliver via `Exec Dashboard – C-Suite – Distribution` (secured SharePoint location / emailed PDF / Power BI, per Requirement 2.7). **Pass gate:** the published one-pager is delivered, the schedule is live, and the first two snapshots confirm trend is accruing. **Rollback:** `-RegisterSchedule -Remove -Commit` stops collection; the page and history are files you retain or delete.

Step 8 — Operate and keep it trustworthy (Expand)

1. Each cycle, re-run `OPSEXEC-Validate.ps1` before distribution so a stale portal value, a lapsed license, or a broken source is caught **before** the executive sees it.

2. Refresh the **portal-only** values (Requirement 2.6) on the agreed cadence and update the read dates; review thresholds with stakeholders as posture matures.

3. Re-run `OPSEXEC-Scope.ps1` periodically so scope drift (new labels/policies/workloads) is folded into the coverage denominators. **Pass gate:** validation is green each cycle and trend is unbroken.

5. Testing & Validation — Microsoft Purview Executive Dashboard

Work through every gate. Each has a **method**, an **expected result (pass)**, and a **fail action**. Tests are read-only — the scripts collect and render; they change no Purview policy. Do not distribute the C-suite one-pager until every gate passes for the pilot draft, then re-confirm on each refresh.

> ***Remember:*** *the dashboard is a* ***reporting layer****. These gates prove the page is* ***accurate, complete, and reconciles to its Purview sources*** *— they do not prove the underlying posture is *adequate* (that is a compliance/records/legal determination), nor do they create history the trend section did not*

yet accrue.

Gate 1 — Sources are reachable and licensing is known

• **Method:** `OPSEXEC-Discovery.ps1` (and `OPSEXEC-Scope.ps1` for permission health).

• **Pass:** each KPI source is classified *reachable-by-API*, *portal-read*, or *not licensed*, with the licensing read from Graph `subscribedSkus`.

• **Fail:** assign the missing read role (Requirement 2.2) or license (2.1); a *not-licensed* source is shown as such on the page, not fabricated.

Gate 2 — The metric definition is complete and approved

• **Method:** open `Exec Metrics – Compliance Signals – Definition`; run `OPSEXEC-Validate.ps1` against it.

• **Pass:** every KPI has a source, an owner, a threshold band, and (for portal-only values) a read date; the in-scope set (workloads/labels/policies/assessments) is named.

• **Fail:** complete the definition (Implementation Steps 2–3) and re-validate.

Gate 3 — The pilot one-pager renders with all three sections

• **Method:** `OPSEXEC-Report.ps1 ... -Stage Pilot` → open `Exec Dashboard – Pilot – Stage1`.

• **Pass:** compliance posture, data risk, and trend sections all render; every KPI shows a value **or** an explicit *"not licensed / portal-read pending,"* each with a data-source and as-of timestamp.

• **Fail:** check the collector output for the empty KPI; confirm the source is reachable (Gate 1) and in the definition (Gate 2).

Gate 4 — Every number reconciles to its Purview source

• **Method:** for each KPI, compare the page value to its portal blade (Compliance Manager, Content Explorer, DLP Alerts/Activity Explorer, retention policies, Insider Risk aggregate, DSPM) — Implementation Step 5.

• **Pass:** each value matches its source within the expected refresh/latency window, or is honestly labeled *"verify licensing/tenant."*

• **Fail:** fix the cause — wrong denominator/scope, stale portal value, a filter difference, or an unlicensed source being counted — and re-render.

Gate 5 — Coverage denominators are correct

• **Method:** confirm label/retention coverage uses the agreed in-scope workloads/labels/policies (Requirement 2.5), via `OPSEXEC-Scope.ps1`.

• **Pass:** the coverage percentages use the defined denominator; adding/removing an in-scope item changes the percentage as expected.

• **Fail:** correct the in-scope set in the definition and re-collect.

Gate 6 — Portal-only values are current (not stale)

• **Method:** `OPSEXEC-Validate.ps1` staleness check against each portal-only value's read date.

• **Pass:** Compliance Manager score, Insider-Risk aggregate, and DSPM recommendations were read within the refresh window and their read dates are recorded.

• **Fail:** the owner re-reads the value from the portal, updates the definition and read date, and re-validates.

Gate 7 — Insider Risk is aggregate and de-identified

• **Method:** inspect the rendered page and the snapshot for any Insider Risk detail.

• **Pass:** only **aggregate counts by severity** appear — **no user names, no case detail**; the Insider Risk source is read under a **separate** role from the dashboard author.

• **Fail:** remove any identifying detail; confirm role separation (Requirement 2.2) before re-rendering.

Gate 8 — Thresholds colour correctly

• **Method:** compare each KPI's colour band to the agreed green/amber/red thresholds in the definition.

• **Pass:** each KPI is coloured against its **agreed** threshold (a low score reads red, good coverage reads green) — the colours reflect the definition, not the tool's opinion.

• **Fail:** correct the threshold band in the definition and re-render.

Gate 9 — Trend accrues across refreshes

• **Method:** run the collection on two (or more) cycles and inspect `Exec Trend – Monthly Snapshot – Baseline`.

• **Pass:** each run appends a dated snapshot; the trend section shows direction across the accrued points (and honestly shows "insufficient history" until enough points exist).

• **Fail:** confirm the scheduled collection is committed (Implementation Step 7) and appending to the trend store, not overwriting.

Gate 10 — Scheduled refresh and distribution work end to end

• **Method:** let `Exec Collector – Purview Signals – Scheduled` run on cadence; confirm `Exec Dashboard – Posture-Risk-Trend – OnePage` reaches `Exec Dashboard – C-Suite – Distribution`.

• **Pass:** the schedule fires, the page rebuilds, validation is green, and the one-pager is delivered to the C-suite channel.

• **Fail:** check the scheduled task, the output path, and the distribution step; re-run `OPSEXEC-Validate.ps1`.

Evidence to retain

- `OPSEXEC-Discovery.ps1` / `OPSEXEC-Scope.ps1` outputs (source + licensing inventory).
- The approved `Exec Metrics – Compliance Signals – Definition` (KPIs, thresholds, scope, portal-value owners).
- The reconciliation notes (Gate 4) showing each number matched to its source.
- `OPSEXEC-Validate.ps1` output per cycle and the accruing snapshots.

*Whether the posture the dashboard reports is **adequate** — a good-enough compliance score, sufficient label/retention coverage, an acceptable risk level — is a **compliance/records/privacy/legal determination**. These gates prove the page is **accurate**, not that the posture is **sufficient**.*

6. Troubleshooting — Microsoft Purview Executive Dashboard

The failure modes that make an executive dashboard **wrong, empty, or stale**, plus the collection and scheduling issues you'll hit. Each: **symptom** → **likely cause** → **fix.** Most "the dashboard is wrong" reports are a denominator, a stale portal value, or a licensing gap — this guide separates those from real faults.

A KPI shows blank / "no data"

- **Cause A — source not licensed.** DSPM, Insider Risk, Content Explorer, or advanced DLP need **E5 / E5 Compliance** (some preview); the KPI can't populate without the plan. **Fix:** `OPSEXEC-Discovery.ps1` reports which plans are present; assign the license (Requirement 2.1) or leave the KPI shown as *"not licensed / verify tenant"* — do not fabricate it.
- **Cause B — missing read role.** The collector account can't read that source. **Fix:** `OPSEXEC-Scope.ps1` reports the permission error; assign the least-privilege **read** role (Requirement 2.2).
- **Cause C — portal-only value not entered.** Compliance Manager score / Insider-Risk aggregate / DSPM recommendations are portal-read in many tenants and weren't filled in. **Fix:** the named owner reads the value from the portal and enters it (with a read date) in the definition (Requirement 2.6).

A number doesn't match the Purview portal

- **Cause A — wrong denominator/scope.** Coverage percentages use the in-scope workloads/labels/policies; a different scope gives a different percentage. **Fix:** confirm the in-scope set with `OPSEXEC-Scope.ps1` and align the definition (Requirement 2.5).
- **Cause B — refresh latency.** Activity/Content Explorer and DLP alerts lag; the portal and the snapshot were taken at different times. **Fix:** note the as-of timestamps; re-collect and compare within the same window.
- **Cause C — filter difference.** The portal blade is filtered (date range, severity, workload) differently from the collector query. **Fix:** match the collector's filter to the portal view (severity, time window, record type).

The compliance score / Insider-Risk aggregate / DSPM count is stale

- **Cause:** these are **portal-only** in many tenants; the owner hasn't re-read them this cycle.
- **Fix:** `OPSEXEC-Validate.ps1` flags any portal-only value older than the refresh window; the owner re-reads it from the portal, updates the value and read date, and re-validates. **Verify whether a supported API is now available in your tenant** before assuming manual is still required.

`Export-ContentExplorerData` / `Export-ActivityExplorerData` errors or returns nothing

- **Cause A — endpoint/preview.** These are **Security & Compliance PowerShell** (`Connect-IPPSSession`) cmdlets and `Export-ContentExplorerData` is **preview/tenant-dependent**. **Fix:** connect with `Connect-IPPSSession`; confirm the cmdlet is available in your tenant (`Get-Command Export-ContentExplorerData`); if absent, treat that KPI as portal-read.
- **Cause B — role.** Content Explorer needs the **Content Explorer List/Content Viewer** roles. **Fix:** assign the reader roles (Requirement 2.2).
- **Cause C — no classified content yet.** A genuinely empty result means little in-scope content is classified — a **real** finding to show, not a bug. **Fix:** report the low coverage; the remedy is labeling/classification, not the dashboard.

Trend section is empty or shows a sudden step-change

- **Cause A — insufficient history.** Trend accrues only from your first collection; two points aren't a trend. **Fix:** let the schedule run several cycles; the page honestly shows "insufficient history" until then.
- **Cause B — snapshot overwritten.** The trend store was replaced instead of appended. **Fix:** confirm `OPSEXEC-Report.ps1` **appends** to `Exec Trend – Monthly Snapshot – Baseline`; never overwrite it.
- **Cause C — definition changed mid-stream.** A KPI or threshold changed, so the line jumps. **Fix:** annotate the break so a definition change isn't misread as a real movement (Best Practices).

The scheduled collection didn't run

- **Cause:** the scheduled task wasn't committed, the account's token expired, or the host was off.
- **Fix:** confirm `OPSEXEC-Configure.ps1 -RegisterSchedule -Commit` created `Exec Collector – Purview Signals – Scheduled`; check the task history and that the collector account can complete a modern-auth/MFA sign-in unattended (or run it interactively on cadence). Re-run `OPSEXEC-Validate.ps1` after any missed cycle.

Insider Risk detail appears on the page (privacy incident)

- **Cause:** the collector or definition included more than aggregate counts, or an over-privileged role read case detail.

• **Fix:** the executive page must carry **aggregate counts by severity only — no names, no case detail**. Remove any identifying data immediately, confirm **role separation** (the Insider Risk reader is not the dashboard author, Requirement 2.2), and re-render. Insider-risk outcomes are **deferred to HR/legal**.

Connect-IPPSSession / Connect-MgGraph fails

• **Cause:** module not installed, or MFA/modern-auth not completed.

• **Fix:** `Install-Module ExchangeOnlineManagement -Scope CurrentUser` and `Install-Module Microsoft.Graph -Scope CurrentUser`; complete the interactive modern-auth sign-in (Requirement 2.3). `OPSEXEC-Scope.ps1` reports which endpoints connected.

Timestamps / percentages look inconsistent across sources

• **Cause:** each Purview source refreshes on its own latency and audit/activity timestamps are **UTC**.

• **Fix:** show a per-KPI as-of timestamp (the collector does), normalize to UTC when correlating, and don't compare two KPIs captured in different windows as if simultaneous.

When to escalate beyond this solution

• **Whether the reported posture is *adequate*** (score, coverage, risk level), **retention-schedule sufficiency, or any regulatory obligation** → **compliance, records, privacy, and legal** functions (the dashboard measures; it does not judge).

• **Insider-risk case handling** → **HR/legal** under the Insider Risk program's governance, not the dashboard.

• **A source genuinely missing despite correct licensing** → verify against current Microsoft documentation and open a Microsoft support case; Purview capabilities, APIs, and blades evolve.

7. Best Practices — An Executive Dashboard the C-Suite Trusts

Operating principles for keeping the Purview executive dashboard accurate, credible, and useful. Grounded in Microsoft's public guidance and the **Assess** → **Design** → **Configure** → **Pilot** → **Validate** → **Expand** model.

Report the truth, don't decorate it

• The dashboard's only job is to be **right**. A single wrong number in front of an executive costs more trust than a dozen correct ones earn — reconcile every value to its Purview source (Testing Gate 4) before anyone senior sees it.

• Where a source isn't licensed or is portal-only, **say so on the page** (*"not licensed / verify tenant," "portal-read as of <date>"*). An honest gap is credible; a fabricated number is a liability. The dashboard shows what you **have**, never what you wish you had.

• It's a **reporting layer, not a control.** If coverage is low, the fix is the underlying capability (labels, DLP, retention), not the report. Don't tune the denominator to make the page look better.

Trend is the executive signal — protect the history

• Executives act on **direction**, not a point-in-time score. The monthly snapshot is the most valuable thing this solution produces over time — **never overwrite** `Exec Trend – Monthly Snapshot – Baseline`; always append.

• Trend only accrues **forward** from your first collection. Start capturing snapshots as early as possible, even before the page is perfect, so the history exists when the board asks "are we improving?"

• Keep the metric **definitions stable** so the trend stays comparable; when you must change a KPI or threshold, annotate the break so a step-change in the line isn't misread as a real movement.

Least-privilege, read-only, and Insider-Risk role separation

• The dashboard author needs **read** across the sources — not write, not investigation access. Use **Global Reader / Compliance Data Administrator**-class roles; the scripts never change a Purview policy.

• **Insider Risk is HR/privacy-sensitive.** Only **aggregate, de-identified** counts belong on an executive page — never names or case detail. The person who reads the Insider Risk aggregate holds a **separate** Insider Risk role from the dashboard author; outcomes are **deferred to HR/legal**.

• Treat the compliance/risk signal itself as sensitive — the dashboard reveals where your weak spots are. Secure the distribution channel and review who receives it.

Thresholds are a governance decision, not a default

• Green/amber/red bands encode what your organization considers **acceptable** — agree them with compliance/records/security stakeholders and record them in the definition. The tool colours against the agreed bands; it doesn't decide what "good" is.

• Revisit thresholds as posture matures: a coverage target that's aspirational at launch should tighten as the program improves, so the page keeps driving the right behaviour.

Respect the licensing and preview reality

• DSPM, Insider Risk, Content Explorer, and advanced DLP are **E5 / E5 Compliance** (some **preview**); Compliance Manager's premium assessments are gated too. Populate a KPI only where its source is genuinely licensed, and **verify the current supported method** before automating a value — Purview's APIs and blades evolve.

• Some values are **portal-only** in many tenants (Compliance Manager score, Insider-Risk aggregate, DSPM recommendations). Keep a named owner and a **read date** for each, and re-check whether a supported API has since become available.

Validate before every distribution

• Run `OPSEXEC-Validate.ps1` each cycle — it catches the stale portal value, the lapsed license, the broken source **before** the executive does. A dashboard that silently goes wrong is worse than no dashboard.

• Keep the reconciliation and validation evidence; when an executive challenges a number, you can show exactly which Purview blade it came from and when.

Keep it a one-pager

• The value is **compression** — three questions, one page. Resist the urge to add every metric Purview can produce; that recreates the portal sprawl the dashboard exists to escape.

• Route anyone who wants the detail to the source blade (or a companion operational report); the executive page stays posture, risk, and trend — nothing more.

8. Deliverables Summary — Microsoft Purview Executive Dashboard

What you receive, what to run, and what "done" looks like.

Documentation

Document	Purpose
1. Solution Overview	What the executive dashboard is, the compliance-posture / data-risk / trend signal stack it summarizes, what you control, honest limitations, and the naming convention.
2. Requirements Guide	Dashboard-specific prerequisites (source licensing, least-privilege read roles incl. Insider-Risk separation, S&C PowerShell + Graph toolchain, metric set & thresholds, in-scope set, portal-read values, render/distribution target, pilot & cadence) — each with Verify / Implement.
3. Implementation Guide	Ordered steps: assess sources, design the metric set/thresholds, configure the definition & schedule, pilot the one-pager, reconcile every number to source, validate, publish, and accrue trend — pilot-first with pass/fail gates and rollback.
5. Testing & Validation	Ten gates from "sources reachable" to "scheduled refresh & distribution," each with method / pass / fail action.
6. Troubleshooting	The failure modes that make a dashboard wrong or empty — unreachable/portal-only sources, stale values, wrong denominators, licensing gaps, trend overwrite, Insider-Risk leakage — and their fixes.

Document	Purpose
7. Best Practices	Operating principles: report the truth, protect the trend history, least-privilege & Insider-Risk separation, thresholds as governance, licensing/preview reality, validate before distribution, keep it a one-pager.

PowerShell tools (5)

All run **read-only or `-WhatIf` by default**; the only writes are the metric definition file and the scheduled task, which require `-Commit`. No credentials or secrets are embedded. Signal collection uses **Security & Compliance PowerShell** (`Connect-IPPSSession`) and **Microsoft Graph**; the dashboard is a **reporting layer** and changes no Purview policy.

Script	Role	Default
`OPSEXEC-Discovery.ps1`	Assess: which KPI sources are reachable-by-API vs portal-only vs not-licensed (labels, DLP, retention, Content/Activity Explorer, Compliance Manager, Insider Risk, DSPM via Graph `subscribedSkus`).	Read-only
`OPSEXEC-Configure.ps1`	Configure: write/update the metric definition (`Exec Metrics – Compliance Signals – Definition`) and register/remove the scheduled collection (`Exec Collector – Purview Signals – Scheduled`).	`-WhatIf` (writes on `-Commit`)
`OPSEXEC-Report.ps1`	Report: collect the signals, merge portal-only values, render the one-page dashboard (HTML + CSV, Power BI-ready), and append the monthly snapshot to the trend store.	Read-only (output files only)
`OPSEXEC-Validate.ps1`	Validate: confirm every defined KPI is present, cross-check recomputable counts, and flag stale/missing values against the refresh window.	Read-only
`OPSEXEC-Scope.ps1`	Scope/health helper: connection/permission/licensing health across sources; list in-scope labels, DLP policies, retention policies, and workloads (coverage denominators).	Read-only

Named artifacts created

• One-pager: `Exec Dashboard – Posture-Risk-Trend – OnePage` (pilot: `Exec Dashboard – Pilot – Stage1`).

• Definition: `Exec Metrics – Compliance Signals – Definition`.

• Scheduled collection: `Exec Collector – Purview Signals – Scheduled`.

• Trend store: `Exec Trend – Monthly Snapshot – Baseline`. Distribution: `Exec Dashboard – C-Suite – Distribution`.

Definition of done

• Every KPI source classified (reachable / portal-read / not-licensed) and the **metric definition** approved with thresholds, in-scope set, and portal-value owners.

• The **pilot one-pager** rendered with all three sections and **every number reconciled** to its Purview source (or honestly labeled *"verify licensing/tenant"*).

• **Validation green** — no missing/stale KPIs; Insider Risk **aggregate/de-identified** with role separation confirmed.

• The **C-suite one-pager** published and delivered via the distribution channel; the **scheduled collection live** and **trend accruing** across at least two snapshots.

Boundary: *this solution delivers an* ***accurate reporting layer****. Whether the posture it reports is* ***adequate*** *— a good-enough compliance score, sufficient coverage, an acceptable risk level — and any* ***regulatory/records/legal determination*** *are* ***deferred to your compliance, records, privacy, and legal functions.*** *Source availability is* ***licensing-/tenant-dependent and evolving — verify against your entitlements and current Microsoft documentation.***

Appendix A — PowerShell Scripts

Included in the accompanying ZIP. Each script's header documents full usage.

OPSEXEC-Configure.ps1

Purpose: Materializes the agreed metric set into "Exec Metrics - Compliance Signals - Definition" (a JSON file: KPIs, sources, thresholds, in-scope denominators, portal-only value owners/read-dates) and optionally registers the scheduled collection task "Exec Collector - Purview Signals - Scheduled". Safe by default: prints what it WOULD write/register; only -Commit changes anything. The only writes are a local file and a scheduled task - no Purview policy is changed, no secrets stored. HOW TO RUN powershell -File .\OPSEXEC-Configure.ps1 -DefinitionPath .\ExecMetrics.json -WhatIf powershell -File .\OPSEXEC-Configure.ps1 -DefinitionPath .\ExecMetrics.json -Commit powershell -File .\OPSEXEC-Configure.ps1 -RegisterSchedule -Cadence Monthly -ReportScript .\OPSEXEC-Report.ps1 -DefinitionPath .\ExecMetrics.json -WhatIf EXPECTED RESULTS -WhatIf prints the definition JSON and/or the schtasks command. -Commit writes the file and/or registers the task. -RegisterSchedule -Remove -Commit unregisters it (rollback). .NOTES Purview category - Executive Dashboard. Writes gated behind -Commit. Keep Insider Risk values AGGREGATE/de-identified. No secrets. Pilot the definition (Stage=Pilot) before scheduling. > [CmdletBinding(DefaultParameterSetName='Definition')] param([Parameter(Mandatory)] [string]$DefinitionPath, [string[]]$InScopeWorkloads = @('Exchange','SharePoint','OneDrive','Teams'), [string]$Cadence = 'Monthly', # Monthly | Weekly [string]$ReportScript, # path to OPSEXEC-Report.ps1 for the scheduled run [switch]$RegisterSchedule, [switch]$Remove, # with -RegisterSchedule -Commit: unregister the task [switch]$Commit)

How to run: powershell -File .\OPSEXEC-Configure.ps1 -DefinitionPath .\ExecMetrics.json -WhatIf powershell -File .\OPSEXEC-Configure.ps1 -DefinitionPath .\ExecMetrics.json -Commit powershell -File .\OPSEXEC-Configure.ps1 -RegisterSchedule -Cadence Monthly -ReportScript .\OPSEXEC-Report.ps1 -DefinitionPath .\ExecMetrics.json -WhatIf EXPECTED RESULTS -WhatIf prints the definition JSON and/or the schtasks command. -Commit writes the file and/or registers the task. -RegisterSchedule -Remove -Commit unregisters it (rollback). .NOTES Purview category - Executive Dashboard. Writes gated behind -Commit. Keep Insider Risk values AGGREGATE/de-identified. No secrets. Pilot the definition (Stage=Pilot) before scheduling. > [CmdletBinding(DefaultParameterSetName='Definition')] param([Parameter(Mandatory)] [string]$DefinitionPath, [string[]]$InScopeWorkloads = @('Exchange','SharePoint','OneDrive','Teams'), [string]$Cadence = 'Monthly', # Monthly | Weekly [string]$ReportScript, # path to OPSEXEC-Report.ps1 for the scheduled run [switch]$RegisterSchedule, [switch]$Remove, # with -RegisterSchedule -Commit: unregister the task [switch]$Commit)

OPSEXEC-Discovery.ps1

Purpose: Connects to Security & Compliance PowerShell and Microsoft Graph and classifies each dashboard KPI source as reachable-by-API, portal-only, or not-licensed: sensitivity labels, DLP policies, retention policies, Content/Activity Explorer, plus Insider Risk / DSPM / advanced-DLP licensing (Graph subscribedSkus). Read-only - changes nothing. Optional CSV export. HOW TO RUN powershell -File .\OPSEXEC-Discovery.ps1 -CsvPath .\Exec-Sources.csv Requires ExchangeOnlineManagement (Connect-IPPSSession) and Microsoft.Graph (licensing). Interactive modern-auth sign-in; no secrets in code. EXPECTED RESULTS A per-source table (Source / Reachable / Detail) and a licensing summary. "CHECK" means verify your licensing/tenant, not a defect. Feeds "Exec Metrics - Compliance Signals - Definition". .NOTES Purview category - Executive Dashboard. Read-only. Sources are

licensing-/tenant-dependent and the portal evolves - some values are portal-only in your tenant. No client names; no secrets. > [CmdletBinding()] param([string]$UserPrincipalName, [string]$CsvPath, [switch]$SkipGraphLicensing)

How to run: powershell -File .\OPSEXEC-Discovery.ps1 -CsvPath .\Exec-Sources.csv Requires ExchangeOnlineManagement (Connect-IPPSSession) and Microsoft.Graph (licensing). Interactive modern-auth sign-in; no secrets in code. EXPECTED RESULTS A per-source table (Source / Reachable / Detail) and a licensing summary. "CHECK" means verify your licensing/tenant, not a defect. Feeds "Exec Metrics - Compliance Signals - Definition". .NOTES Purview category - Executive Dashboard. Read-only. Sources are licensing-/tenant-dependent and the portal evolves - some values are portal-only in your tenant. No client names; no secrets. > [CmdletBinding()] param([string]$UserPrincipalName, [string]$CsvPath, [switch]$SkipGraphLicensing)

OPSEXEC-Report.ps1

Purpose: Reads "Exec Metrics - Compliance Signals - Definition" (from OPSEXEC-Configure.ps1), collects the API-readable KPIs (label/retention coverage, DLP/activity counts) via Security & Compliance PowerShell, uses the portal-entered values (with their read dates) for portal-only KPIs, RAG-scores each against its thresholds, and renders the one-page "Exec Dashboard - One Pager - <Stage>" (HTML + CSV). It then APPENDS the cycle's snapshot to "Exec Trend - Monthly Snapshot - Baseline" so trend accrues over time. Read-only against Purview - the only writes are the report files and the trend store. Insider-risk KPIs are carried as AGGREGATE counts by severity only (no names, no case detail). HOW TO RUN powershell -File .\OPSEXEC-Report.ps1 -DefinitionPath .\ExecMetrics.json -Stage Pilot powershell -File .\OPSEXEC-Report.ps1 -DefinitionPath .\ExecMetrics.json -Stage Publish Requires ExchangeOnlineManagement (Connect-IPPSSession). -TrendPath overrides the trend store path; -OutDir sets where the one-pager lands. Interactive modern-auth; no secrets. EXPECTED RESULTS An HTML one-pager + a CSV of KPI/value/RAG, and one appended row in the trend store. PASS = every KPI shows either a collected value or an explicit "not licensed / portal-read - verify" - never a fabricated number - and the trend store grew by exactly one snapshot. .NOTES Purview category - Executive Dashboard. Read-only vs Purview. Trend store is APPEND-ONLY - never overwrite it. Keep Insider Risk AGGREGATE/de-identified. Sources are licensing-/tenant-dependent; some are portal-only. No client names; no secrets. > [CmdletBinding()] param([Parameter(Mandatory)][string]$DefinitionPath, [ValidateSet('Pilot','Publish')][string]$Stage = 'Pilot', [string]$UserPrincipalName, [string]$TrendPath = ".\Exec-Trend-Monthly-Snapshot-Baseline.json", [string]$OutDir = ".")

How to run: powershell -File .\OPSEXEC-Report.ps1 -DefinitionPath .\ExecMetrics.json -Stage Pilot powershell -File .\OPSEXEC-Report.ps1 -DefinitionPath .\ExecMetrics.json -Stage Publish Requires ExchangeOnlineManagement (Connect-IPPSSession). -TrendPath overrides the trend store path; -OutDir sets where the one-pager lands. Interactive modern-auth; no secrets. EXPECTED RESULTS An HTML one-pager + a CSV of KPI/value/RAG, and one appended row in the trend store. PASS = every KPI shows either a collected value or an explicit "not licensed / portal-read - verify" - never a fabricated number - and the trend store grew by exactly one snapshot. .NOTES Purview category - Executive Dashboard. Read-only vs Purview. Trend store is APPEND-ONLY - never overwrite it. Keep Insider Risk AGGREGATE/de-identified. Sources are licensing-/tenant-dependent; some are portal-only. No client names; no secrets. > [CmdletBinding()] param([Parameter(Mandatory)][string]$DefinitionPath, [ValidateSet('Pilot','Publish')][string]$Stage = 'Pilot', [string]$UserPrincipalName, [string]$TrendPath = ".\Exec-Trend-Monthly-Snapshot-Baseline.json", [string]$OutDir = ".")

OPSEXEC-Scope.ps1

Purpose: Verifies the collector account can read each dashboard source and lists the candidate in-scope set that forms the coverage denominators: published sensitivity labels, DLP policies, retention policies, and the workloads they target. Reports a clear permission/connection error per source rather than failing the whole run. Read-only - changes nothing. HOW TO RUN powershell -File .\OPSEXEC-Scope.ps1 -CsvPath .\Exec-Scope.csv Requires ExchangeOnlineManagement (Connect-IPPSSession). Interactive modern-auth; no secrets. EXPECTED RESULTS A health table (source / OK|ERROR / detail) and an in-scope inventory you confirm and record in "Exec Metrics - Compliance Signals - Definition" as the coverage denominators. .NOTES Purview category - Executive Dashboard. Read-only. Keep Insider Risk roles SEPARATE from the dashboard-author role. No client names; no secrets. > [CmdletBinding()] param([string]$UserPrincipalName, [string]$CsvPath)

How to run: powershell -File .\OPSEXEC-Scope.ps1 -CsvPath .\Exec-Scope.csv Requires ExchangeOnlineManagement (Connect-IPPSSession). Interactive modern-auth; no secrets. EXPECTED RESULTS A health table (source / OK|ERROR / detail) and an in-scope inventory you confirm and record in "Exec Metrics - Compliance Signals - Definition" as the coverage denominators. .NOTES Purview category - Executive Dashboard. Read-only. Keep Insider Risk roles SEPARATE from the dashboard-author role. No client names; no secrets. > [CmdletBinding()] param([string]$UserPrincipalName, [string]$CsvPath)

OPSEXEC-Validate.ps1

Purpose: Read-only quality gate for the executive dashboard. Checks "Exec Metrics - Compliance Signals - Definition" and the latest "Exec Trend - Monthly Snapshot - Baseline" entry for the faults that make an exec page WRONG, EMPTY, or STALE: every KPI has a source + thresholds; every in-scope denominator is set; every portal-only value has an owner, a value, and a read date INSIDE the refresh window; the Insider-Risk KPI is AGGREGATE-only (no name/case fields); and the trend store is present and append-only (monotonic timestamps). Read-only - changes nothing. HOW TO RUN powershell -File .\OPSEXEC-Validate.ps1 -DefinitionPath .\ExecMetrics.json -TrendPath .\Exec-Trend-Monthly-Snapshot-Baseline.json Optional: -OutPath to write the PASS/FAIL/WARN results to CSV. EXPECTED RESULTS A per-check PASS/FAIL/WARN table. PASS overall = no FAIL (WARN allowed with a recorded exception). A portal-only value older than refreshWindowDays is flagged for re-read before publish. No number is ever fabricated - an unlicensed/portal-unavailable source is a documented "verify", not a failure. .NOTES Purview category - Executive Dashboard. Read-only. Enforces privacy-by-design on Insider Risk (aggregate only). No secrets. Verify source availability in your tenant. > [CmdletBinding()] param([Parameter(Mandatory)][string]$DefinitionPath, [string]$TrendPath = ".\Exec-Trend-Monthly-Snapshot-Baseline.json", [string]$OutPath)

How to run: powershell -File .\OPSEXEC-Validate.ps1 -DefinitionPath .\ExecMetrics.json -TrendPath .\Exec-Trend-Monthly-Snapshot-Baseline.json Optional: -OutPath to write the PASS/FAIL/WARN results to CSV. EXPECTED RESULTS A per-check PASS/FAIL/WARN table. PASS overall = no FAIL (WARN allowed with a recorded exception). A portal-only value older than refreshWindowDays is flagged for re-read before publish. No number is ever fabricated - an unlicensed/portal-unavailable source is a documented "verify", not a failure. .NOTES Purview category - Executive Dashboard. Read-only. Enforces privacy-by-design on Insider Risk (aggregate only). No secrets. Verify source availability in your tenant. > [CmdletBinding()] param([Parameter(Mandatory)][string]$DefinitionPath, [string]$TrendPath = ".\Exec-Trend-Monthly-Snapshot-Baseline.json", [string]$OutPath)